

ACTIVIDAD PRÁCTICA

Registro de ventas con archivos en Python

Del diccionario venta al archivo ventas.txt

Producto esperado: Un programa que registre ventas, las convierta en líneas de un archivo, las lea nuevamente y calcule un resumen.

1. Contexto del problema

Una feria necesita registrar las ventas realizadas durante la jornada. La información debe permanecer disponible aunque el programa se cierre, por lo que cada venta se guardará en un archivo llamado ventas.txt.

Cada venta tendrá los siguientes datos:

- Nombre del cliente.
- Producto comprado.
- Valor unitario en pesos chilenos.
- Cantidad de unidades.
- Total de la venta.
- Fecha de la venta.

2. El diccionario venta

Antes de guardar la información en el archivo, cada venta se representará mediante un diccionario:

```
venta = {  
    "nombre_cliente": "María González",  
    "producto": "Pan amasado",  
    "valor_unitario": 2500,  
    "cantidad": 2,  
    "total": 5000,  
    "fecha": "2026-08-06"  
}
```

Importante: El total no se solicita al usuario: se calcula con `valor_unitario * cantidad`. Los montos son enteros porque trabajaremos con pesos chilenos sin fracciones.

Clave	Tipo	Ejemplo
nombre_cliente	str	María González
producto	str	Pan amasado
valor_unitario	int	2500
cantidad	int	2
total	int	5000
fecha	str	2026-08-06

3. Formato del archivo ventas.txt

Cada venta se guardará en una línea. Los campos se separarán usando punto y coma (;).

```
María González;Pan amasado;2500;2;5000;2026-08-06  
Juan Pérez;Café;1800;1;1800;2026-08-06
```

El orden de los campos será siempre:

1. nombre_cliente
2. producto
3. valor_unitario
4. cantidad
5. total
6. fecha

Cuidado: No uses punto y coma dentro del nombre del cliente o del producto, porque ese carácter separa los campos.

4. Funciones que debes construir

No escribas todo en un solo bloque. Construye y prueba estas funciones una por una:

Función	Recibe	Debe hacer
solicitar_venta()	Nada	Solicita datos y retorna un diccionario.
calcular_total(valor, cantidad)	Dos enteros	Retorna valor * cantidad.
guardar_venta(venta)	Un diccionario	Agrega una línea al archivo.
leer_ventas()	Nada	Retorna una lista de diccionarios.
mostrar_ventas(ventas)	Una lista	Muestra los registros.
calcular_resumen(ventas)	Una lista	Calcula cantidad y total.

5. Pistas para construir las funciones

5.1. Solicitar y validar una venta

- Solicita nombre del cliente y producto con input().
- Repite la pregunta si alguno está vacío.
- Solicita valor unitario y cantidad como enteros.
- Valida que ambos sean mayores que cero.
- Calcula total con valor_unitario * cantidad.
- Obtén la fecha con datetime o usa un formato definido.
- Retorna un diccionario con las seis claves.

```
def solicitar_venta():  
    # solicitar y validar datos  
    # calcular total  
    # construir y retornar el diccionario  
    pass
```

5.2. Guardar el diccionario

La función debe recibir el diccionario y escribir sus valores en el orden definido. Usa modo a para agregar sin borrar.

```
def guardar_venta(venta):  
    with open("ventas.txt", "a", encoding="utf-8") as archivo:  
        # escribir campos separados por ;  
        pass
```

5.3. Leer el archivo y reconstruir diccionarios

- Abre el archivo en modo r.
- Recorre cada línea con un ciclo for.
- Elimina el salto de línea con strip().
- Separa campos con split(';').
- Convierte los tres montos/cantidades a int.
- Construye un diccionario por línea y agrégalo a una lista.
- Retorna la lista de ventas.

```
def leer_ventas():
    ventas = []
    with open("ventas.txt", "r", encoding="utf-8") as archivo:
        for linea in archivo:
            # limpiar, separar y convertir datos
            pass
    return ventas
```

5.4. Calcular el resumen

Recorre la lista de diccionarios y usa acumuladores. No vuelvas a solicitar los datos al usuario.

```
def calcular_resumen(ventas):
    cantidad_ventas = 0
    total_general = 0

    for venta in ventas:
        # aumentar cantidad
        # sumar venta["total"]
        pass
```

6. Programa principal

Una secuencia mínima para probar el programa puede ser:

7. Solicitar y guardar tres ventas.
8. Leer las ventas desde ventas.txt.
9. Mostrar las ventas recuperadas.
10. Mostrar cantidad y total general.

```
for i in range(3):
    venta = solicitar_venta()
    guardar_venta(venta)

ventas = leer_ventas()
mostrar_ventas(ventas)
calcular_resumen(ventas)
```

Nota: El bloque anterior solo coordina las funciones. Debes completar cada función y decidir qué valores retornará.

7. Validaciones obligatorias

- Cliente no vacío.
- Producto no vacío.
- Valor unitario entero mayor que cero.
- Cantidad entera mayor que cero.
- Total calculado, no ingresado manualmente.
- Uso de `with open(...)`.
- Las ventas nuevas no borran las anteriores.

8. Pruebas mínimas

Prueba	Entrada	Resultado esperado
Venta válida	2500 y 2	Total igual a 5000
Monto inválido	0 o negativo	Se vuelve a solicitar
Cantidad inválida	0 o negativa	Se vuelve a solicitar
Datos vacíos	cliente o producto vacío	Se vuelve a solicitar
Tres ventas	Tres registros	Tres líneas y resumen correcto

9. Producto final

Entrega una carpeta que contenga:

- `programa_ventas.py`
- `ventas.txt` generado por el programa.
- Evidencia de ejecución con tres ventas registradas.

Criterio de logro: El programa crea diccionarios de ventas, calcula el total, guarda cada venta, recupera los registros y muestra un resumen correcto.